oom-runner — User Manual

Memory-bounded systemd scopes for any workload
System post-mortem 2026-04-28
2026-04-28

Overview

`oom-runner.sh` wraps **almost any command** in its own bounded **systemd transient scope** (foreground) or **service** (background) so a single runaway can never starve the rest of your session. It is **layer 2** of the three-layer toolkit:

1. **`oom-hardening.sh`** sets the **umbrella** — bounds for the whole user slice plus enables `systemd-oomd`.
2. **`oom-runner.sh`** *(this script)* sets the **per-workload bound** — Claude, MCP servers, Android builds, browsers, containers, IDEs, ad-hoc shell pipelines. Each runs in its own transient cgroup with its own MemoryMax / MemoryHigh / TasksMax / CPUQuota.
3. **`oom-watch/`** is the **forensic eye** — a Go daemon that samples atop and writes a Markdown report (with full `/proc/<pid>/cmdline`, PPID, parent's cmdline, cgroup path, peak RSS) **before** thresholds breach. When a workload wrapped by `oom-runner` is the leak source, the report's forensic-detail section names it by full argv and shows its cgroup path under `user.slice/.../oomrun-*.{scope,service}` — so you know not just *which process* but *which oom-runner unit* was involved. Install with `sudo make oomwatch-deploy`; see `manuals/oom-watch-deployment-guide.md` and `oom-watch-runbook.md`.

Defaults are deliberately generous (`MemoryMax=12G` if you set nothing) so wrapping commands rarely breaks them. The actual win is: **you can no longer exhaust the whole machine from inside one scope**.

The script needs **no root**. It uses your per-user systemd manager.

Quick Start

```
# Anything that takes -- and a command works
oom-runner -- claude
oom-runner -- yandex-browser
oom-runner -- python heavy.py

# Use a preset for sane workload-sized limits
oom-runner --preset claude  -- claude
oom-runner --preset mcp     -n upstash -- npm exec @upstash/context7-mcp@latest
oom-runner --preset build   -- bash -c 'cd ~/proj && m -j8'
oom-runner --preset browser -- yandex-browser

# Run a shell pipeline (use --shell to keep pipes/redirects working)
oom-runner --shell -- 'cat huge.log | grep ERROR | head -100'

# Background a service
oom-runner -d --preset mcp -n my-mcp -- npm exec my-mcp@latest
oom-runner list                # see all active oom-runner units
oom-runner status my-mcp       # show its limits + memory usage
oom-runner logs   my-mcp -f    # tail its journal
oom-runner kill   my-mcp       # stop it
oom-runner clean               # stop ALL oom-runner units
```

Add a shell alias for convenience:

```
# ~/.bashrc or ~/.zshrc
alias oom='~/Downloads/oom-runner.sh'
alias claude='~/Downloads/oom-runner.sh --preset claude -- claude'
```

How it works

When you run:

```
oom-runner --preset claude -- claude
```

the script invokes:

```
exec systemd-run --user --collect --scope \
    --unit=oomrun-claude-<rand>.scope \
    --pty \
    --description='oom-runner: claude (memmax=10G, memhigh=8G, swap=2G, cpu=200%)' \
    -p MemoryMax=10G \
    -p MemoryHigh=8G \
    -p MemorySwapMax=2G \
    -p TasksMax=4096 \
    -p CPUQuota=200% \
    -p IOWeight=200 \
    -p Environment=PATH=… -p Environment=DISPLAY=… …  \
    -- claude
```

`systemd-run` creates a **transient cgroup** at:

```
/sys/fs/cgroup/user.slice/user-1000.slice/user@1000.service/app.slice/oomrun-claude-<rand>.scope
```

The kernel enforces every resource limit on that cgroup. When the command exits, `--collect` removes the unit automatically. When the command exceeds `MemoryMax`, the kernel kills processes **inside that cgroup only** — your shell, IDE, other Claude instances, etc. all survive.

Subcommands

Command	Effect
`oom-runner [opts] -- <cmd> [args]`	Run `cmd` in a bounded foreground scope
`oom-runner run [opts] -- <cmd> [args]`	Same, explicit subcommand form
`oom-runner -d [opts] -- <cmd> [args]`	Run `cmd` as a bounded background service
`oom-runner list`	List active `oomrun-*` scopes/services
`oom-runner status <unit>`	Show one unit’s status (incl. live mem usage)
`oom-runner logs <unit> [...]`	`journalctl --user -u <unit>` (extra args pass through, e.g. `-f`, `-n 100`)
`oom-runner kill <unit>`	Stop one unit (sends SIGTERM, then SIGKILL after timeout)
`oom-runner clean`	Stop **all** oomrun units
`oom-runner presets`	List available presets and their limits
`oom-runner --help`	Full usage
`oom-runner --version`	Print script version

`<unit>` may be the bare custom suffix (e.g. `upstash`), the prefixed name (e.g. `oomrun-upstash`), or the full unit name with `.scope` / `.service`.

Run options

Option	Default	Effect
`-p, --preset NAME`	`default`	Apply a named preset. See **Presets** below.
`-m, --memory-max SIZE`	preset or 12G	Hard memory ceiling. Refused below 128 MiB without `--yes`.
`-h, --memory-high SIZE`	preset	Soft throttle. Auto-clamped to `≤ MemoryMax`.
`-s, --memory-swap-max SIZE`	preset	Swap cap.
`-t, --tasks-max N`	preset or `infinity`	Max tasks (PIDs) in the cgroup.
`-c, --cpu-quota PCT`	unset	CPU quota; `200` = 2 cores.
`--io-weight N`	preset	I/O priority weight 1–10000.
`-n, --name NAME`	random	Custom unit suffix. Becomes `oomrun-NAME`.
`-d, --detach`	off	Run as background `--service` instead of foreground `--scope`.
`--shell`	off	Combine remaining args into a single `bash -c` line. Lets you use pipes, redirects, here-strings, env expansions.
`--pty` / `--no-pty`	auto	Force/disable PTY allocation (foreground only). Default: PTY when stdout is a terminal.
`--setenv K=V`	(none)	Pass env var into the unit. Repeatable.
`--inherit-env REGEX`	see below	Inherit env vars matching the regex from caller.
`--no-inherit-env`	off	Start with a clean env (no inherited vars).
`--workdir PATH`	`$PWD`	Working directory inside the unit.
`--nice N`	0	Nice level (-20 .. 19).
`--read-only` / `--rw`	rw	

Option	Default	Effect
		Mount root read-only/read-write inside unit.
`--no-protect`	off	Disable extra hardening directives (only relevant for `-d`).
`-v, --verbose`	off	Print the systemd-run command before exec.
`-y, --yes`	off	Skip below-floor confirmation.
`--dry-run`	off	Print what would run; do not execute.

Default `--inherit-env` regex

Matches the env vars that 99% of GUI / CLI / dev workloads actually need:

```
^(PATH|HOME|LANG|LC_.*|TERM|DISPLAY|WAYLAND_DISPLAY|XDG_.*|DBUS_.*|SSH_AUTH_SOCK|EDITOR|GIT_.*)$
```

Override with `--inherit-env '<your-regex>'` or disable entirely with `--no-inherit-env`. Inherited variables are passed via `-p Environment=…`, not by attaching to the host environment, which is what makes the unit hermetic.

Safety guards

Guard	What it does
128 MiB floor	Refuses `MemoryMax` below 128 MiB unless `--yes`. Prevents accidentally crippling a process to the point of instant death.
Auto-clamp `MemoryHigh`	If you set `MemoryHigh > MemoryMax`, the script clamps `MemoryHigh = MemoryMax` and warns. This is silently allowed by systemd, but indicates a config bug.
Command existence check	If the command isn't in `$PATH` and isn't an executable file, prints a warning before launch (still tries — systemd will report the real error).
Pre-flight: user manager reachable	Verifies `XDG_RUNTIME_DIR` and that `systemctl --user` works. Falls back to `/run/user/$(id -u)`.
Pre-flight: slice accounting	Warns if `MemoryAccounting=no` on `user-X.slice` (limits still work in the scope, but the umbrella isn't accounted).
systemd version check	Warns if systemd < 240.
Hardening on `-d` services	Sets `NoNewPrivileges=yes`, `PrivateTmp=yes`, `ProtectSystem=full` on detached services. Disable with `--no-protect`.
Auto cleanup	`--collect` ensures units are removed from systemd state after exit (no zombie unit clutter).

Presets

Run `oom-runner presets` to print the live table. Defaults as of v1.1:

Preset	MemoryMax	Memory-High	Swap-Max	Tasks-Max	CPUQuota	IOWeight
`default`	12G	10G	2G	∞	–	–
`safe`	8G	6G	1G	4096	–	–
`tiny`	512M	384M	128M	256	–	–
`small`	1G	768M	256M	512	–	–
`medium`	4G	3G	512M	2048	–	–
`large`	16G	12G	2G	∞	–	–
`huge`	32G	24G	4G	∞	–	–
`claude`	10G	8G	2G	4096	–	200
`mcp`	2G	1500M	512M	1024	–	100
`build`	32G	24G	6G	∞	–	400
`browser`	8G	6G	1G	4096	–	150
`container`	4G	3G	512M	2048	–	100
`editor`	6G	5G	1G	2048	–	200
`vm`	16G	12G	2G	∞	–	100
`shell`	4G	3G	512M	2048	–	–

Explicit `-m` / `-h` / `-s` / `-c` / `--io-weight` flags **override** preset values. Combine freely:

```
# claude preset, but raise to 16G, narrow CPU to 1 core
oom-runner --preset claude -m 16G -h 14G -c 100 -- claude
```

Common scenarios

Wrapping Claude Code

Make every Claude session a 10G island:

```
# In ~/.bashrc
alias claude='~/Downloads/oom-runner.sh --preset claude -- claude'
```

Each new Claude process gets its own scope. MCP servers spawned **by** Claude inherit the cgroup automatically — they all share the 10G ceiling.

Wrapping individual MCP servers

When MCP servers are launched separately (e.g. via a wrapper script):

```
oom-runner --preset mcp -n upstash      -- npm exec @upstash/context7-mcp@latest
oom-runner --preset mcp -n playwright   -- npm exec @playwright/test@latest
oom-runner --preset mcp -n helius       -- npm exec helius@latest
```

Now any one MCP eating 4G doesn’t starve the others.

Android builds

```
oom-runner --preset build -- bash -c '
    cd ~/proj/Android_15
    source build/envsetup.sh
    lunch aosp_arm64-userdebug
    m -j8
'
```

`build` preset = 32G hard / 24G soft / 6G swap / 400% CPU (4 cores). If the build’s working set explodes, only the build dies; you don’t lose your IDE, browser, terminals, or running services.

Browsers

```
oom-runner --preset browser -- yandex-browser
oom-runner --preset browser -m 4G -- chromium --user-data-dir=/tmp/chr
```

A 4G ceiling per browser instance is generally enough; below that you’ll see tab eviction, which is preferable to OOM-killing the rest of your system.

Shell pipelines

The `--shell` flag combines remaining args into one `bash -c` invocation:

```
oom-runner --shell -- '
    set -e
    cat large.json |
        jq ".records[] | select(.size > 1e6)" |
        head -1000 > /tmp/big.json
'
```

Detached background workers

```
oom-runner -d --preset mcp -n redis-mcp -- npm exec @upstash/redis-mcp@latest
oom-runner logs redis-mcp -f       # tail
oom-runner status redis-mcp        # check memory usage
oom-runner kill redis-mcp          # stop
```

A detached service is fully managed: systemd restarts it on reboot only if you create a permanent unit (this script's units are transient — they disappear on logout). For permanent units, use a custom user unit.

One-off Python scripts

```
oom-runner -m 4G -- python my_script.py
```

Or with custom env:

```
oom-runner -m 8G \
    --setenv PYTHONUNBUFFERED=1 \
    --setenv MY_API_KEY="$MY_API_KEY" \
    -- python my_script.py
```

Containers

The script wraps the **launcher** (e.g. `podman run`), not the container's runtime. Most container engines obey their own resource flags — combine:

```
oom-runner --preset container -- \
    podman run --rm -m 1g --cpus=1 my-image
```

The cgroup limits enforce a hard maximum even if `podman -m` is bypassed.

Verification

```
# Run a small task and inspect the live cgroup
oom-runner -m 256M -- sleep 30 &
oom-runner list

# In another terminal:
oom-runner status <unit-from-list>
# Look for:
#   Memory: 224K (high: 256M, max: 256M, swap max: …, available: …, peak: …)
#                              ^ this is the kernel-enforced ceiling
```

Or read the cgroup file directly:

```
cat /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/app.slice/
        <unit>.scope/memory.max
```

A pure stress test:

```
oom-runner -m 256M -- bash -c '
    python3 -c "x=bytearray(); _=[x.extend(b\"a\"*1024*1024) for _ in range(2000)]"
'
# Process is killed inside the scope, not your shell.
```

Troubleshooting

“User systemd manager is not reachable”

You’re in an SSH session without lingering. Either enable lingering once, then re-login:

```
sudo loginctl enable-linger $(id -un)
```

…or run the script from inside a graphical login session.

Foreground command exits but my terminal looks weird

`--pty` allocates a TTY for the scope. If the command does its own terminal manipulation (full-screen TUIs), you might end up with leftover terminal state. Reset with `reset` or `stty sane`. Or run with `--no-pty` if your command doesn’t need a TTY.

`npm exec` / Node command can't find packages

Node looks for `node_modules` relative to `$PWD`. Foreground scopes inherit `$PWD` from the shell, so this works. Detached services do **not** inherit `$PWD` from the shell — you must pass `--workdir`:

```
oom-runner -d --workdir ~/myproj -- npm exec something
```

"Unknown assignment: WorkingDirectory=…"

You're on a very old systemd that doesn't support `WorkingDirectory` on transient units. Upgrade systemd, or accept that detached units run with `/` as cwd.

`--shell` mode interpreted my `$VAR` wrong

The shell in `--shell` mode is `/bin/bash -c "$line"` — `$line` is expanded **inside** that bash, not in your calling shell. To pass a value from the caller, use `--setenv`:

```
MY_VAR=hello oom-runner --shell --setenv "MY_VAR=$MY_VAR" -- 'echo $MY_VAR'
```

Process gets OOM-killed at MemoryHigh, not MemoryMax

That's normal. `MemoryHigh` is a soft throttle — when crossed, the kernel becomes very aggressive about reclaiming memory and the process can be slowed to the point of being non-responsive. If you want the process to get more headroom before reclaim, raise `MemoryHigh`. If you want it to be killed sooner, lower `MemoryMax`.

`oom-runner list` doesn't show my unit

Either the unit already exited (and `--collect` cleaned it up), or it crashed before becoming "active". Check journal:

```
journalctl --user -t systemd | grep oomrun-
```

Detached service immediately exits

Foreground commands that read stdin won't work as `-d` services unless their stdin is redirected. Provide one explicitly or use `--shell`:

```
oom-runner -d --shell -- 'exec my-command < /dev/null > /tmp/out 2>&1'
```

“Refusing MemoryMax=64M (< 128M floor)”

Add `--yes`:

```
oom-runner -m 64M --yes -- echo "really small"
```

The floor exists because tiny limits cripple modern processes (a single Node.js startup easily allocates 80 MiB). Override consciously.

Multiple Claude instances inherit the same cgroup

When Claude is launched from inside another Claude process (e.g. via a shell-out), the child inherits the parent’s cgroup — the 10G ceiling applies to **both combined**. To bound them separately, launch each from a fresh shell:

```
oom-runner --preset claude -n claude-A -- claude
# in another terminal
oom-runner --preset claude -n claude-B -- claude
```

FAQ

Q. Does this work for GUI apps? Yes. The default `--inherit-env` includes `DISPLAY`, `WAYLAND_DISPLAY`, `XDG_*`, `DBUS_*`, etc. GUI apps launch normally inside the scope.

Q. Will systemd kill my command if I just close the terminal? Foreground scopes are tied to your shell (the caller). Closing the terminal sends SIGHUP to the scope’s processes. Detached services (`-d`) run independently of the calling terminal — they survive logout only if `Linger=yes` (see `loginctl enable-linger`).

Q. Can I limit GPU memory too? No — cgroup v2 doesn’t expose GPU memory. For Nvidia, use `CUDA_VISIBLE_DEVICES`. For AMD, use the GPU’s own controls. For Intel, the iGPU shares system RAM and is bounded by your `MemoryMax`.

Q. How is this different from `ulimit`? `ulimit` is per-process and easy to bypass (a child fork can re-raise). Cgroups are per-cgroup and **enforced by the kernel** regardless of how many processes you fork inside.

Q. How is this different from running `nice` / `ionice`? Those tune scheduling priority. They don’t bound memory. They’re useful **inside** the scope (via `--nice`) but don’t replace `MemoryMax`.

Q. Can I nest oom-runners? Yes. Cgroup v2 enforces “no resource on a child can exceed the parent’s” automatically. So:

```
oom-runner -m 16G -- bash -c '
    oom-runner -m 4G -- python a.py &
    oom-runner -m 4G -- python b.py &
    wait
'
```

…works, with the inner scopes capped at 4G each within the 16G outer.

Q. Does the script create permanent units? No — every unit is **transient**: it lives only until the command exits, then `--collect` removes it. For permanent user services use `systemctl --user edit --user --force --full <name>.service` directly.

Q. Can I run as root for system-wide bounds? This script targets `--user`. For system-wide use, swap `--user` for `--system` in `systemd-run` and edit accordingly. The user-scope use case is by far the most common and is the post-mortem fix recommended by `Crash_Report.md`.

Companion Tools

- **`oom-hardening.sh`** — sets the umbrella limits and enables `systemd-oomd`. **Apply this first**, then start using `oom-runner`.
- **`Crash_Report.md`** — the 2026-04-28 incident this whole toolkit was built to prevent.

Appendix A — Full output of `presets`

```
PRESET       Mem.Max    Mem.High   Swap.Max     TasksMax   CPUQuota   IOWeight
------------------------------------------------------------------------------
default      12G        10G        2G           infinity               -
safe         8G         6G         1G           4096                   -
tiny         512M       384M       128M         256                    -
small        1G         768M       256M         512                    -
medium       4G         3G         512M         2048                   -
large        16G        12G        2G           infinity               -
huge         32G        24G        4G           infinity               -
claude       10G        8G         2G           4096                   200
mcp          2G         1500M      512M         1024                   100
build        32G        24G        6G           infinity               400
browser      8G         6G         1G           4096                   150
container    4G         3G         512M         2048                   100
editor       6G         5G         1G           2048                   200
vm           16G        12G        2G           infinity               100
shell        4G         3G         512M         2048                   -
```

Appendix B — Exit codes

The foreground form (default) `exec`s `systemd-run`, so the exit code of `oom-runner -- cmd ...` is the exit code of `cmd`. Pre-flight failures exit 1 before launching.

The detached form returns 0 once `systemd-run` has accepted the unit. Use `oom-runner status <unit>` and `oom-runner logs <unit>` to inspect.

Appendix C — Cgroup paths

Foreground scope:

```
/sys/fs/cgroup/user.slice/user-<UID>.slice/user@<UID>.service/app.slice/oomrun-<name>-
<rand>.scope
```

Detached service:

```
/sys/fs/cgroup/user.slice/user-<UID>.slice/user@<UID>.service/app.slice/oomrun-
<name>.service
```

Inspect:

```
cd /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/app.slice/oomrun-
        claude-*.scope/
ls
cat memory.current memory.max memory.high memory.swap.max
cat cgroup.procs
```